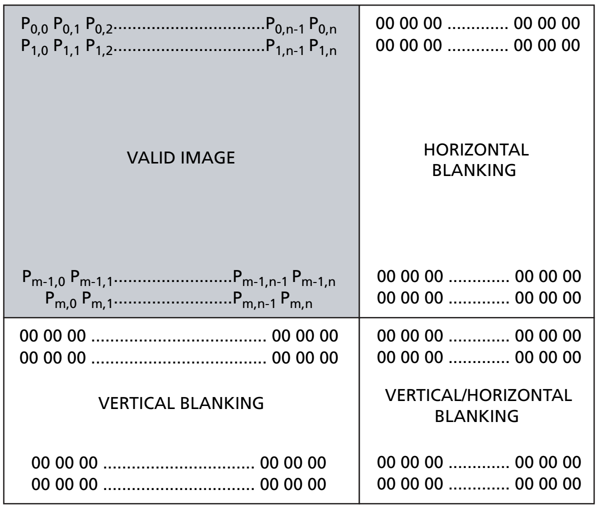
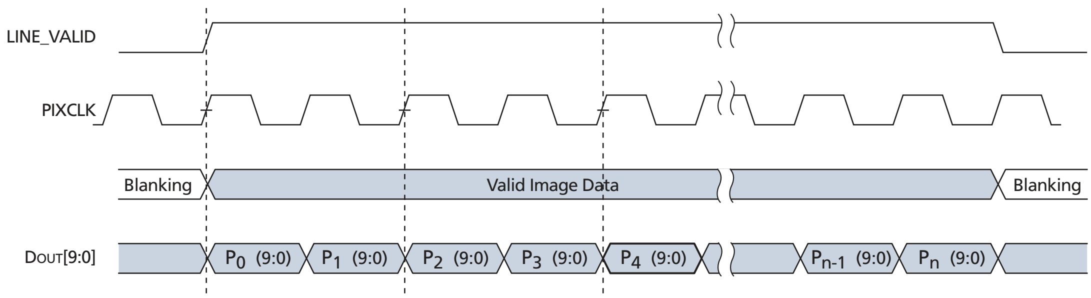
**Figure 15: Valid Image Data**




## Raw Data Timing

The sensor core output data is synchronized with the PIXCLK output. When LV is HIGH, one pixel's data is output on the 10-bit DOUT output bus every PIXCLK period. By default, the PIXCLK signal runs at the same frequency as the master clock, and its falling edges occur one-half of a master clock period after transitions on LV, FV, and DOUT[9:0] (see Figure 16). This allows PIXCLK to be used as a clock to sample the data. PIXCLK is continuously enabled, even during the blanking period.

**Figure 16: Pixel Data Timing Example**

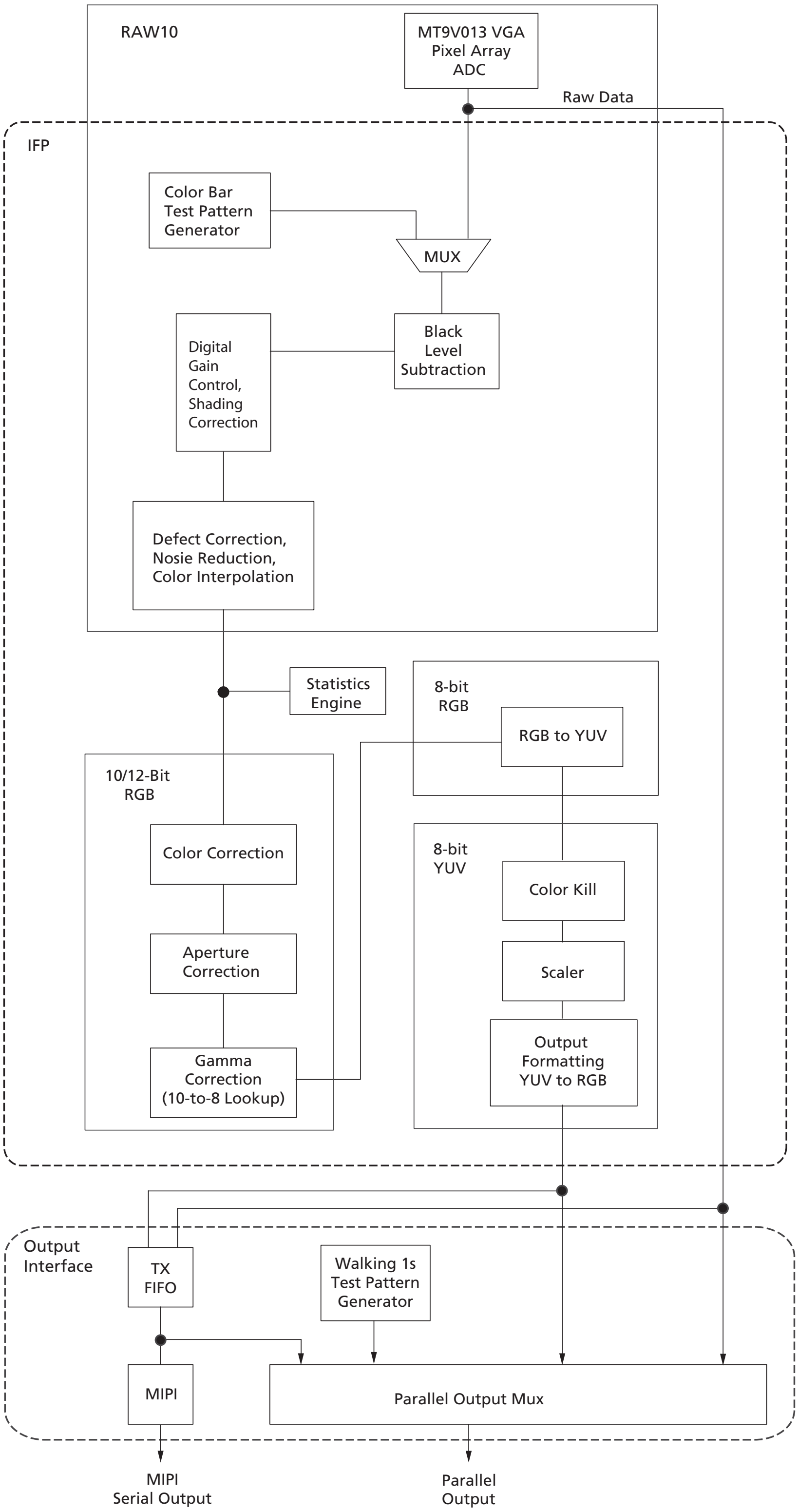


# SOC Description

## Image Flow Processor

Image and color processing in the MT9M113 is implemented as an IFP coded in hardware logic. The IFP can be controlled by registers from the outside but during normal operation, the embedded microcontroller will automatically adjust the operation parameters. The IFP is broken down into different sections, as outlined in Figure 17 on page 23.

**Figure 17: Image Flow Processor**



## Test Patterns

During normal operation of the MT9M113, a stream of raw image data from the sensor core is continuously fed into the color pipeline. For test purposes, this stream can be replaced with a fixed image generated by a special test module in the pipeline. The module provides a selection of test patterns sufficient for basic testing of the pipeline.

Test patterns are accessible by programming a register and are shown in Figure 18. Disabling the MCU is recommended before enabling test patterns.

**Figure 18: Color Bar Test Pattern**

| Test Pattern | Example |
|---|---|
| Flat Field |  |
| Vertical Ramp |  |
| Color Bar |  |
| Vertical Stripes |  |
| Pseudo-Random |  |

## First Black Level Subtraction and Digital Gain

Image stream processing starts with black level subtraction and multiplication of all pixel values by a programmable digital gain. Both operations can be independently set to separate values for each color channel (R, Gr, Gb, B). Independent color channel digital gain can be adjusted with registers. Independent color channel black level adjustments can also be made. If the black level subtraction produces a negative result for a particular pixel, the value of this pixel is set to "0."

## Shading Correction (SC)

Lenses tend to produce images whose brightness is significantly attenuated near the edges. There are also other factors causing fixed pattern signal gradients in images captured by image sensors. The cumulative result of all these factors is known as image shading. The MT9M113 has an embedded shading correction module that can be programmed to counter the shading effects on each individual R, Gb, Gr, and B color signal.

### The Correction Function

Color-dependent solutions are calibrated using the sensor, lens system, and an image of an evenly illuminated, featureless grey calibration field. From the resulting image, the color correction functions can be derived.

The correction functions can then be applied to each pixel value to equalize the response across the image as follows:

$$P_{corrected}(row,col) = P_{sensor}(row,col) * f(row,col) \qquad \text{(EQ 3)}$$

where $P$ are the pixel values and $f$ is the color dependent correction functions for each color channel.

## Defect Correction and Noise Reduction

The IFP performs continuous defect correction that can mask pixel array defects such as high dark-current (hot) pixels and pixels that are darker or brighter than their neighbors due to photoresponse nonuniformity. The module is edge-aware with exposure that is based on configurable thresholds. The thresholds are changed continuously based on the brightness of the current scene. Noise reduction can be enabled and disabled and thresholds can be set through register settings.

### Color Interpolation and Edge Detection

In the raw data stream fed by the sensor core to the IFP, each pixel is represented by a 10-bit integer number, which can be considered proportional to the pixel's response to a one-color light stimulus, red, green, or blue, depending on the pixel's position under the color filter array. Initial data processing steps, up to and including the defect correction, preserve the one-color-per-pixel nature of the data stream, but after the defect correction it must be converted to a three-colors-per-pixel stream appropriate for standard color processing. The conversion is done by an edge-sensitive color interpolation module. The module pads the incomplete color information available for each pixel with information extracted from an appropriate set of neighboring pixels. The algorithm used to select this set and extract the information seeks the best compromise between preserving edges and filtering out high frequency noise in flat field areas. The edge threshold can be set through register settings.

## Color Correction and Aperture Correction

To achieve good color fidelity of the IFP output, interpolated RGB values of all pixels are subjected to color correction. The IFP multiplies each vector of three pixel colors by a 3 x 3 color correction matrix. The three components of the resulting color vector are all sums of three 10-bit numbers. Since such sums can have up to 12 significant bits, the bit width of the image data stream is widened to 12-bits per color (36-bits per pixel). The color correction matrix can be either programmed by the user or automatically selected by the auto white balance (AWB) algorithm implemented in the IFP. Color correction should ideally produce output colors that are independent of the spectral sensitivity and color crosstalk characteristics of the image sensor. The optimal values of the color correction matrix elements depend on those sensor characteristics and on the spectrum of light incident on the sensor. The color correction variables can be adjusted through register settings.

To increase image sharpness, a programmable 1D or 2D aperture correction (sharpening filter) is applied to color-corrected image data. The gain and threshold for 1D and 2D correction can be defined through register settings.

## Image Cropping

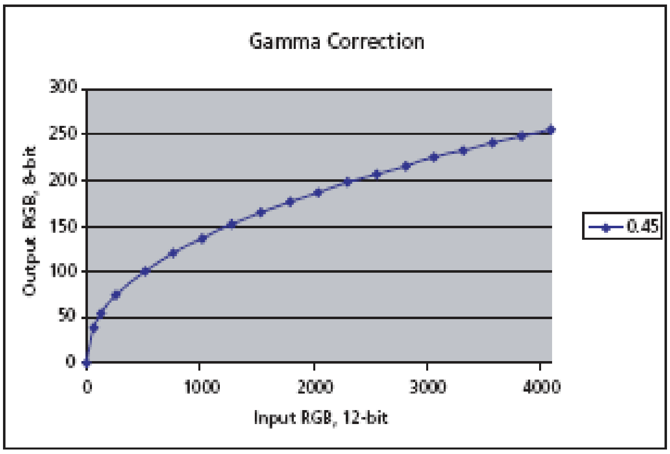
By configuring the cropped and output windows to various sizes, different zooming levels–for example 4X, 2X, and 1X–can be achieved. The location of the cropped window is also configurable so that panning is also supported. A separate cropped window is defined for context A and context B. In both contexts, the height and width definitions for the output window must be equal to or smaller than the cropped image.

## Gamma Correction

The gamma correction curve (as shown in Figure 19 on page 27) is implemented as a piecewise linear function with 19 knee points, taking 12-bit arguments and mapping them to 8-bit output. The abscissas of the knee points are fixed at 0, 64, 128, 256, 512, 768, 1024, 1280, 1536, 1792, 2048, 2304, 2560, 2816, 3072, 3328, 3584, 3840, and 4096. The 8-bit ordinates are programmable through IFP registers.

The MT9M113 IFP includes a block for gamma correction that can adjust its shape based on brightness to enhance the performance under certain lighting conditions. Two custom gamma correction tables may be uploaded, one corresponding to a brighter lighting condition, the other one corresponding to a darker lighting condition. At power-up, the IFP loads the two tables with default values. The final gamma correction table used depends on the brightness of the scene and can take the form of either uploaded tables or an interpolated version of the two tables. A single (non-adjusting) table for all conditions can also be used.

**Figure 19: Gamma Correction Curve**




## Special Effects

Special effects like negative image, sepia, or black and white can be applied to the data stream at this point. These effects can be enabled and selected by registers.

## RGB to YUV Conversion

For further processing, the data is converted from RGB color space to YUV color space.

## Color Kill

To remove high or low light color artifacts, a color kill circuit is included. It affects only pixels whose luminance exceeds a certain preprogrammed threshold. The U and V values of those pixels are attenuated proportionally to the difference between their luminance and the threshold.

## YUV Color Filter

As an optional processing step, noise suppression by one-dimensional low-pass filtering of Y and/or UV signals is possible. A 3- or 5-tap filter can be selected for each signal.

## Image Scaling

To ensure that the size of images output by the MT9M113 can be tailored to the needs of all users, the IFP includes a scaler module. When enabled, this module performs rescaling of incoming images—shrinks them to arbitrarily selected width and height without reducing the field of view and without discarding any pixel values.

The scaler performs pixel binning—divides each input image into rectangular bins corresponding to individual pixels of the desired output image, averages pixel values in these bins, and assembles the output image from the bin averages. Pixels lying on bin boundaries contribute to more than one bin average; their values are added to bin-wide sums of pixel values with fractional weights. The entire procedure preserves all image information that can be included in the downsized output image and filters out high frequency features that could cause aliasing.

The image cropping and scaler module can be used together to implement a digital zoom and pan. If the scaler is programmed to output images smaller than images coming from the sensor core, zoom effect can be produced by cropping the latter from their maximum size down to the size of the output images. The ratio of these two sizes determines the maximum attainable zoom factor. For example, a 1280 x 1024 image rendered on a 160 x 128 display can be zoomed up to eight times, since 1280/160 = 1024/128 = 8. Panning effect can be achieved by fixing the size of the cropping window and moving it around the pixel array.

Due to the loss of sharpness due to pixel binning during image scaling, 1D aperture correction may be applied to increase image sharpness.

## YUV-to-RGB/YUV Conversion and Output Formatting

The YUV data stream emerging from the scaling module can either exit the color pipeline as-is or be converted before exit to an alternative YUV or RGB data format.

## Color Conversion Formulas

### Y'U'V'

This conversion is BT 601 scaled to make YUV range from 0 through 255. This setting is recommended for JPEG encoding and is the most popular, although it is not well defined and often misused in various operating systems.

$$Y' = 0.299 \times R' + 0.587 \times G' + 0.114 \times B' \quad \text{(EQ 4)}$$

$$U' = 0.564 \times (B' - Y') + 128 \quad \text{(EQ 5)}$$

$$V' = 0.713 \times (R' - Y') + 128 \quad \text{(EQ 6)}$$

There is an option where 128 is not added to U'V'.

### Y'Cb'Cr' Using sRGB Formulas

The MT9M113 implements the sRGB standard. This option provides YCbCr coefficients for a correct 4:2:2 transmission.

**Note:** *16 < Y601< 235; 16 < Cb < 240; 16 < Cr < 240; and 0 < = RGB < = 255*

$$Y' = (0.2126 \times R' + 0.7152 \times G' + 0.0722 \times B') \times (219/256) + 16 \quad \text{(EQ 7)}$$

$$Cb' = 0.5389 \times (B' - Y') \times (224/256) + 128 \quad \text{(EQ 8)}$$

$$Cr' = 0.635 \times (R' - Y') \times (224/256) + 128 \quad \text{(EQ 9)}$$

### Y'U'V' Using sRGB Formulas

Similar to the previous set of formulas, but has YUV spanning a range of 0 through 255.

$$Y' = 0.2126 \times R' + 0.7152 \times G' + 0.0722 \times B' \quad \text{(EQ 10)}$$

$$U' = 0.5389 \times (B' - Y') + 128 = -0.1146 \times R - 0.3854 \times G + 0.5 \times B \quad \text{(EQ 11)}$$

$$V' = 0.635 \times (R' - Y') + 128 = 0.5 \times R - 0.4542 \times G - 0.0458 \times B \quad \text{(EQ 12)}$$

There is an option to disable adding 128 to U'V'. The reverse transform is as follows:

$$R' = Y + 1.5748 \times V \quad \text{(EQ 13)}$$

$$G' = Y - 0.1873 \times (U - 128) - 0.4681 \times (V - 128) \quad \text{(EQ 14)}$$

$$B' = Y + 1.8556 \times (U - 128) \quad \text{(EQ 15)}$$

## Output Interface

### Parallel and MIPI Output

The user can select to use either the serial MIPI output or the 8-bit parallel output to transmit the data. Only one of the output modes can be used at any time.

The parallel output can be used with an output FIFO whose memory is shared with the MIPI output FIFO to retain a constant pixel output clock independent from the scaling factor.

When scaling the image or skipping lines, the data would be generated in bursts and the pixel clock would turn on and off in intervals, which might lead to EMI problems. The output FIFO will group all active pixel data together so the pixel clock can be run at a constant speed.

The MIPI output transmitter implements a serial differential sub-LVDS transmitter capable of up to 480 Mb/s. It supports multiple formats, error checking, and custom short packets.

**Table 6: Data Formats Supported by MIPI Interface**

| Data Format | Data Type |
|---|---|
| YUV 422 8-bit | 0x1E |
| RGB565 | 0x22 |
| RGB555 | 0x21 |
| RGB444 | 0x20 |
| RAW8 | 0x2A |
| RAW10 | 0x2B |

Notes: 1. Data will be packed as RAW8 if the data type specified does not match any of the above data types.

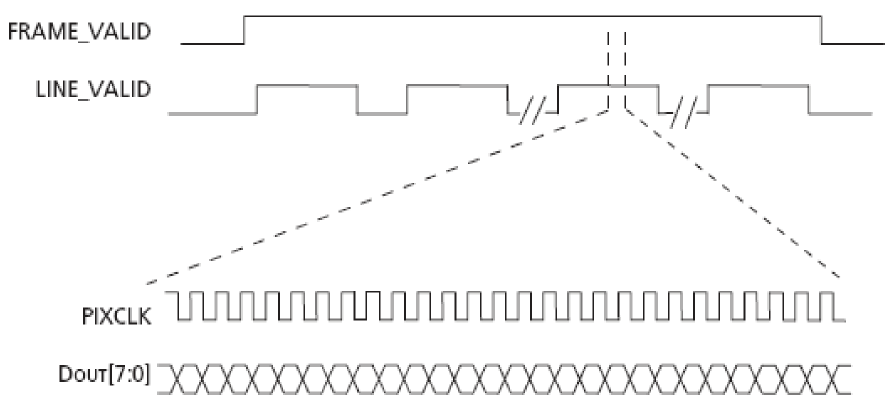
## Output Format and Timing

### YUV/RGB Uncompressed Output

Uncompressed YUV or RGB data can be output either directly from the output formatting block or through a FIFO buffer with a capacity of 800 bytes. This size is large enough to hold one fourth of an uncompressed line at full resolution. Buffering of data is a way to equalize the data output rate when image scaling is used. Scaling produces an intermittent data stream consisting of short high-rate bursts separated by idle periods. High pixel clock frequency during bursts may be undesirable due to EMI concerns.

Figure 20 depicts the output timing of uncompressed YUV/RGB when a scaled data stream is equalized by buffering or when no scaling takes place. The pixel clock frequency remains constant during each LV high period. Scaled data is output at a lower frequency than full size frames, which helps to reduce EMI.

**Figure 20: Timing of Uncompressed Full Frame Data or Scaled Data Passing Through the FIFO**




### Uncompressed YUV/RGB Data Ordering

The MT9M113 supports swapping YCrCb mode, as illustrated in Table 7.

**Table 7: YCrCb Output Data Ordering**

| Mode | Data Sequence | | | |
|---|---|---|---|---|
| Default (no swap) | $Cb_i$ | $Y_i$ | $Cr_i$ | $Y_{i+1}$ |
| Swapped CrCb | $Cr_i$ | $Y_i$ | $Cb_i$ | $Y_{i+1}$ |
| Swapped YC | $Y_i$ | $Cb_i$ | $Y_{i+1}$ | $Cr_i$ |
| Swapped CrCb, YC | $Y_i$ | $Cr_i$ | $Y_{i+1}$ | $Cb_i$ |

The RGB output data ordering in default mode is shown in Table 8. The odd and even bytes are swapped when luma/chroma swap is enabled. R and B channels are bitwise swapped when chroma swap is enabled.

**Table 8: RGB Ordering in Default Mode**

| Mode (Swap Disabled) | Byte | $D_7D_6D_5D_4D_3D_2D_1D_0$ |
|---|---|---|
| RGB 565 | Odd | $R_7R_6R_5R_4R_3G_7G_6G_5$ |
| | Even | $G_4G_3G_2B_7B_6B_5B_4B_3$ |

**Table 8: RGB Ordering in Default Mode (continued)**

| Mode (Swap Disabled) | Byte | $D_7D_6D_5D_4D_3D_2D_1D_0$ |
|---|---|---|
| RGB 555 | Odd | 0 $R_7R_6R_5R_4R_3G_7G_6$ |
| | Even | $G_4G_3G_2B_7B_6B_5B_4B_3$ |
| RGB 444x | Odd | $R_7R_6R_5R_4G_7G_6G_5G_4$ |
| | Even | $B_7B_6B_5B_4$ 0 0 0 0 |
| RGB x444 | Odd | 0 0 0 0 $R_7R6_6R_5R_4$ |
| | Even | $G_7G_6G_5G_4B_7B_6B_5B_4$ |

## Uncompressed 10-Bit Bypass Output

Raw 10-bit Bayer data from the sensor core can be output in bypass mode in two ways:

1. Using 8 data output pads (DOUT[7:0]) and GPIO[1:0]. The GPIO pads are the lowest two bits of data.
2. Using only 8 pads (DOUT[7:0]) and a special 8 + 2 data format, shown in Table 9.

**Table 9: 2-Byte RGB Format**

| Byte | Bits Used | Bit Sequence |
|---|---|---|
| Odd bytes | 8 data bits | $D_9D_8D_7D_6D_5D_4D_3D_2$ |
| Even bytes | 2 data bits + 6 unused bits | 0 0 0 0 0 0 $D_1D_0$ |

## FIFO

During normal pipeline operation, the output data rate is determined by a number of factors: input image size, degree of scaling, and sensor operation mode. As these parameters change during normal sensor operation, output frequency changes. This output frequency may generate RF noise, interfering with the mobile device. By using an output FIFO to maintain a constant output clock frequency, noise is easily filtered out.

The FIFO accumulates data and after a certain number of bytes are stored it will output them in a single burst, making sure that the data rate within the burst remains constant. This approach utilizes a free-running clock, thus making possible minimal RF interference.

# Camera Control

## General Purpose I/Os

The five GPIOs of the MT9M113 can be configured in multiple ways. Each of the I/Os can be used as a simple input/output that can be programmed from the host. The status of the GPIO is read at power up and can be used as a module ID to separate different module suppliers. In addition, module ID can be stored in the one-time programmable memory of the sensor. Information on the OTP memory can be found in "One-Time Programmable Memory."

If 10-bit RAW output is required, GPIO[1:0] can be configured as bit 1 and bit 0 (the LSBs) of a 10-bit data bus.

GPIO[4] can be configured to output a flash pulse to trigger an external Xenon or LED flash or a shutter pulse to control an external shutter.

GPIO[2] can also be configured as inputs to be used as an OE_BAR signal for the data bus.

The general purpose inputs are enabled or disabled through register settings. Once enabled, all five inputs must be driven to valid logic levels by external signals. The state of the general purpose inputs can be read from a register.

## Output Enable Control

When the parallel pixel data interface is enabled, its signals can be switched asynchronously between being driven and High-Z under signal or register control.

## One-Time Programmable Memory

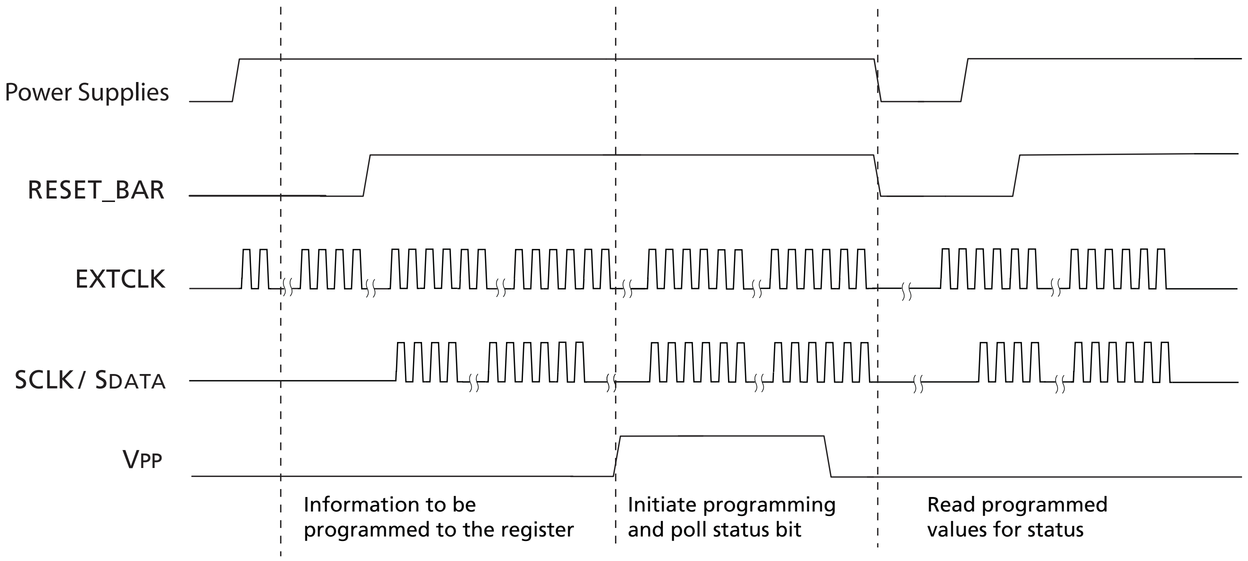
The MT9M113 contains two bytes of OTP memory, suitable for storing module identification that can be programmed during the module manufacturing process. Programming the OTP memory requires the use of a high voltage (7–8V) at the VPP pin. During normal operation, the VPP pin should be left floating. The OTP memory can be accessed through the two-wire serial interface.

### *Programming the OTP Memory*

The programming of the OTP memory requires the sensor to be fully powered and remain in software standby with its clock input applied. The steps below describe the programming process:

1. Apply power to all the power rails of the sensor (VDD, VDD_IO, VAA, VAA_PIX, and so on).
2. Provide the EXTCLK clock input.
3. Perform the proper reset sequence to the sensor.
4. Place the sensor in software standby (sensor default state upon power-up).
5. Write the information to be programmed to the Program_Value register through the two-wire serial interface.
6. Supply a high voltage (7-8V) to the VPP pin.
7. Write to the Program_Value register to indicate the high voltage is applied at the VPP pin in order to start the programming process.
8. Poll the status bit from the Program_Value register for the completion of the programming process.
9. Remove the high voltage at VPP pin.
10. Verify the data at the OTP memory by performing a normal read procedure.

**Figure 21: Sequence of Signals for OTP Memory Operation**



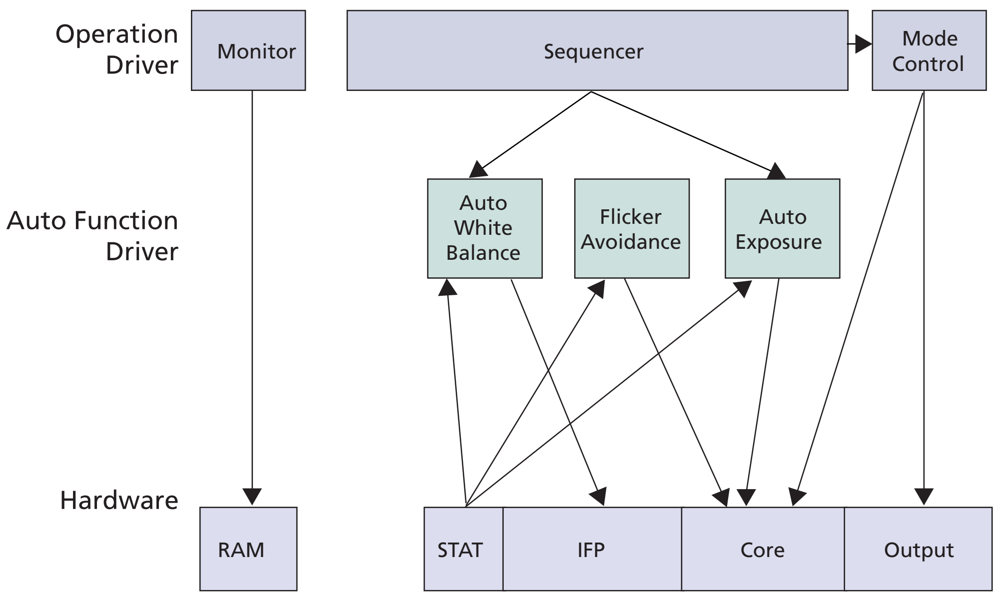

### *Reading the OTP Memory*

The content of the OTP memory can be read through the two-wire serial interface, and it is available for access after power-up.

## Firmware Architecture

The firmware for the MT9M113 is implemented in multiple drivers that are responsible for different parts of operation.

**Figure 22: Software Architecture**




## Sequencer

The sequencer is responsible for coordinating all events triggered by the user. It is implemented as a state machine. For example, sending a capture command to the sequencer will change the resolution from preview to full resolution, turn on or off an external LED, and switch back to preview after capturing the frame. The user can define the sensor setup for preview and capture.

## Context and Operational Modes

The MT9M113 can operate in several modes including preview, still capture (snapshot), and video. All modes of operation are individually configurable and are organized as two contexts—context A and context B. Context switching can be accomplished by sending a command through the two-wire serial interface.

### Preview Mode

Context A is primarily intended for use in the preview mode. During preview, the sensor usually outputs low resolution images at a relatively high frame rate, and its power consumption is kept to a minimum.

### Still Capture and Video Modes

Context B can be configured for the still capture or video mode, as required by the user. For still capture configuration, the user typically specifies the desired output image size, if flash should be enabled, how many frames to capture, and so forth. For video, the user might select a different image size and a fixed frame rate.

### Snapshot and Flash

To take a snapshot, the user must send a command that changes the context from A to B. Typical sequence of events after this command are:

1. The camera may turn on its LED flash, if it has one and is required to use it. With the flash on, the camera exposure and white balance are automatically adjusted to the changed illumination of the scene.
2. The camera captures one or more frames of desired size. A camera equipped with a Xenon flash strobes while capturing images. When capturing images is completed, the camera automatically returns to context A and resumes running in preview mode.

**Note:** This sequence of events can take up to 10 frames.

### Video

To start video capture, the user must change relevant context B settings, such as capture mode, image size and frame rate, and again send a context change command. Upon receiving it, the MT9M113 switches to the modified context B settings, while continuing to output YUV-encoded image data. AE adjusts automatically and provides a smooth continuous operation. To exit the video capture mode, the user must send another context change command causing the sensor to switch back to context A.

## Auto Exposure

The auto exposure algorithm performs automatic adjustments of the image brightness by controlling exposure time and analog gains of the sensor core as well as digital gains applied to the image.

Auto exposure is implemented by a firmware driver that analyzes image statistics collected by the exposure measurement engine, makes a decision, and programs the sensor core and color pipeline to achieve the desired exposure. The measurement engine subdivides the image into 16 windows organized as a 4 x 4 grid.

Two auto exposure algorithm modes are available:

1. Average brightness tracking (ABT)
2. Dynamic range tracking (DRT)

The average brightness tracking AE uses a constant average tracking algorithm where a target brightness value is compared to a current brightness value, and the gain and integration time are adjusted accordingly to meet the target requirement.

The dynamic range tracking AE examines the data from the statistics engine and produces a corrected brightness target so that overexposed or underexposed scene can be avoided. This also allows a manual control of the image brightness. Both algorithms are available in preview and capture modes.

## AE Driver

This exposure mode is activated during preview. This mode can also be enabled during video capture mode. It relies on the statistics engine that tracks speed and amplitude of the change of the overall luminance in the selected windows of the image.

Backlight compensation is achieved by weighting the luminance in the center of the image higher than the luminance on the periphery. Other algorithm features include the rejection of fast fluctuations in illumination (time averaging), control of speed of response, and control of the sensitivity to the small changes. While the default settings are adequate in most situations, the user can program target brightness, measurement window, and other parameters described above.

The driver calculates image brightness based on average luma values received from 16 programmable equal-size rectangular windows forming a 4 x 4 grid. In preview mode, 16 windows are combined in two segments: central and peripheral. The central segment includes 4 central windows. All remaining windows belong to peripheral segment. Scene brightness is calculated as average luma in each segment taken with certain weights.

The driver changes AE parameters (integration time, gains, and so on) to drive brightness to the programmable target. The value of the single step approach to the target value can be controlled.

To avoid unwanted reaction of AE on small fluctuations of scene brightness or momentary scene changes, the AE driver uses a temporal filter for luma and a threshold around the AE luma target. The driver changes AE parameters only if the buffered luma is larger than the AE target step and pushes the luma beyond the threshold.

## Evaluative Algorithm

A scene evaluative AE algorithm is available for use in snapshot mode. The algorithm performs scene analysis and classification with respect to its brightness, contrast, and composure and then decides to increase, decrease, or keep original exposure target. It makes most difference for backlight and bright outdoor conditions.

## Accelerated Settling During Overexposure

The AE speed is direction-dependent. Transitioning from oversaturation to target can take more time than transitioning from undersaturation. The AE driver has a mode that speeds up AE for overexposed scenes.

The AE driver counts the number of AE windows whose average brightness is equal to or greater than some value, 250 by default. For a scene having saturated regions, the average luma is underestimated due to signal clipping. The driver compensates underestimation by a factor that can be defined.

## Exposure Control

To achieve the required amount of exposure, the AE driver adjusts the sensor integration time, gains, ADC reference, and IFP digital gains. In addition, a variable is available for the user to adjust the overall brightness of the scene. To reject flicker, integration time is typically adjusted in increments of steps. The incremental step specifies the duration in row times equal to one flicker period. Thus, flicker is rejected if integration time is kept a natural factor of the flicker period.

## Auto White Balance

The MT9M113 has a built-in AWB algorithm designed to compensate for the effects of changing spectra of the scene illumination on the quality of the color rendition. The algorithm consists of two major parts: a measurement engine performing statistical analysis of the image and a driver performing the selection of the optimal color correction matrix, digital, and sensor core analog gains. While default settings of these algorithms are adequate in most situations, the user can reprogram base color correction matrices, place limits on color channel gains, and control the speed of both matrix and gain adjustments.

## Flicker Detection

Flicker occurs when the integration time is not an integer multiple of the period of the light intensity. The automatic flicker detection block does not compensate for the flicker, but rather avoids it by detecting the flicker frequency and adjusting the integration time. For integration times below the light intensity period (10ms for 50Hz environment), flicker cannot be avoided.

Flicker shows as horizontal bars rolling up or down. The MCU looks for these rolling bars using a thin horizontal window, which outputs luma average and is applied to 48 points in the upper half of the image. The MCU repeats the same sampling on the next frame and subtracts 48 samples from previous frame from corresponding samples from the current frame. Skipping more frames between subtraction can be set by a variable. The MCU then smooths the 48 sampling points, applies an amplitude threshold to avoid false detection, and looks at the resulting waveform. If flicker is present, the waveform should have a frequency within the search range. Assuming the flicker power is a sine wave, subtracting two frames results in:

$$sin(wt) - sin(wt + a) = 2 \times sin(a/2) \times cos(wt + (a/2)) \qquad \text{(EQ 16)}$$

which is a cosine wave of the original frequency.

# Two-Wire Serial Interface

The two-wire serial interface bus enables READ and WRITE access to control and status registers within the MT9M113. This interface is designed to be compatible with the MIPI Alliance Standard for Camera Serial Interface 2 (CSI-2) 1.0, which uses the electrical characteristics and transfer protocols of the two-wire serial interface specification.

The interface protocol uses a master/slave model in which a master controls one or more slave devices. The sensor acts as a slave device. The master generates a clock (SCLK) that is an input to the sensor and used to synchronize transfers.

Data is transferred between the master and the slave on a bidirectional signal (SDATA). SDATA is pulled up to VDD_IO off-chip by a 1.5KΩ resistor. Either the slave or master device can drive SDATA LOW—the interface protocol determines which device is allowed to drive SDATA at any given time.

## Protocol

Data transfers on the two-wire serial interface bus are performed by a sequence of low-level protocol elements, as follows:

1. a (repeated) start condition
2. a slave address/data direction byte
3. a 16-bit register address
4. an (a no) acknowledge bit
5. two data bytes
6. a stop condition

The bus is idle when both SCLK and SDATA are HIGH. Control of the bus is initiated with a start condition, and the bus is released with a stop condition. Only the master can generate the start and stop conditions.

### Start Condition

A start condition is defined as a HIGH-to-LOW transition on SDATA while SCLK is HIGH. At the end of a transfer, the master can generate a start condition without previously generating a stop condition; this is known as a "repeated start" or "restart" condition.

### Stop Condition

A stop condition is defined as a LOW-to-HIGH transition on SDATA while SCLK is HIGH.

### Data Transfer

Data is transferred serially, 8 bits at a time, with the MSB transmitted first. Each byte of data is followed by an acknowledge bit or a no-acknowledge bit. This data transfer mechanism is used for the slave address and data direction byte and for message bytes.

One data bit is transferred during each SCLK clock period. SDATA can change when SCLK is low and must be stable while SCLK is HIGH.

### Slave Address/Data Direction Byte

Bits [7:1] of this byte represent the device slave address and bit [0] indicates the data transfer direction. A "0" in bit [0] indicates a WRITE, and a "1" indicates a READ. The default slave addresses used by the MT9M113 are 0x78 (WRITE address) and 0x79 (READ address). Alternate slave addresses of 0x7A (WRITE address) and 0x7B (READ address) can be selected by asserting the SADDR input signal.

### Message Byte

Message bytes are used for sending register addresses and register WRITE data to the slave device and for retrieving register READ data. The protocol used is outside the scope of the two-wire serial interface specification.

### Acknowledge Bit

Each 8-bit data transfer is followed by an acknowledge bit or a no-acknowledge bit in the SCLK clock period following the data transfer. The transmitter (which is the master when writing, or the slave when reading) releases SDATA. The receiver indicates an acknowledge bit by driving SDATA LOW. As for data transfers, SDATA can change when SCLK is LOW and must be stable while SCLK is HIGH.

### No-Acknowledge Bit

The no-acknowledge bit is generated when the receiver does not drive SDATA LOW during the SCLK clock period following a data transfer. A no-acknowledge bit is used to terminate a READ sequence.

## Typical Serial Transfer

A typical READ or WRITE sequence begins by the master generating a start condition on the bus. After the start condition, the master sends the 8-bit slave address/data direction byte. The last bit indicates whether the request is for a READ or a WRITE, where a “0” indicates a WRITE and a “1” indicates a READ. If the address matches the address of the slave device, the slave device acknowledges receipt of the address by generating an acknowledge bit on the bus.

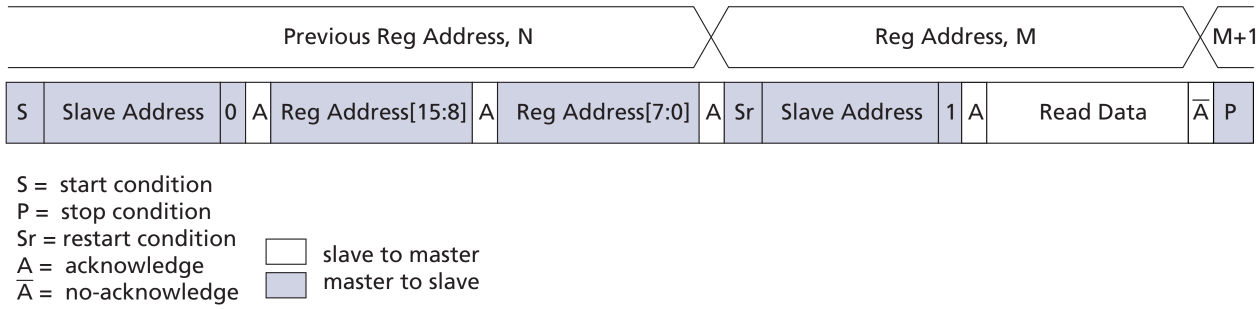
If the request was a WRITE, the master then transfers the 16-bit register address to which a WRITE should take place. This transfers takes place as two 8-bit sequences and the slave sends an acknowledge bit after each sequence to indicate that the byte has been received. The master then transfers the data as an 8-bit sequence; the slave sends acknowledge bit at the end of the sequence. After 8 bits have been transferred, the slave’s internal register address is incremented automatically, so that the next 8 bits are written to the next register address. The master stops writing by generating a (re)start or stop condition.

If the request was a READ, the master sends the 8-bit WRITE slave address/data direction byte and 16-bit register address, just as in the WRITE request. The master then generates a (re)start condition and the 8-bit READ slave address/data direction byte, and clocks out the register data, eight bits at a time. The master generates an acknowledge bit after each 8-bit transfer. The slave’s internal register address is incremented after every 8 bits are transferred. The data transfer is stopped when the master sends a no-acknowledge bit.

## Single READ from Random Location

This sequence (see Figure 23 on page 40) starts with a dummy WRITE to the 16-bit address that is to be used for the READ. The master terminates the WRITE by generating a restart condition. The master then sends the 8-bit READ slave address/data direction byte and clocks out one byte of register data. The master terminates the READ by generating a no-acknowledge bit followed by a stop condition. Figure 23 shows how the internal register address maintained by the MT9M113 is loaded and incremented as the sequence proceeds.

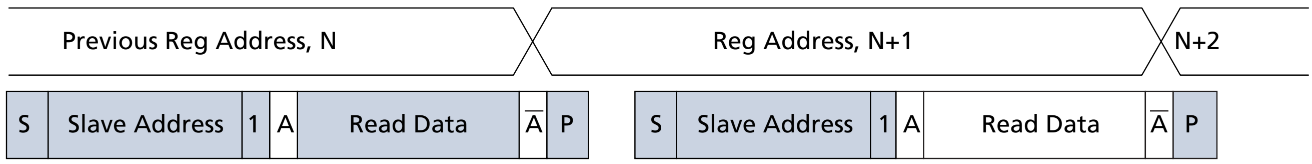
**Figure 23: Single READ from Random Location**




## Single READ from Current Location

This sequence (Figure 24) performs a READ using the current value of the MT9M113 internal register address. The master terminates the READ by generating a no-acknowledge bit followed by a stop condition. The figure shows two independent READ sequences.

**Figure 24: Single Read from Current Location**




## Sequential READ, Start from Random Location

This sequence (Figure 25) starts in the same way as the single READ from random location (Figure 23). Instead of generating a no-acknowledge bit after the first byte of data has been transferred, the master generates an acknowledge bit, and continues to perform byte reads until "L" bytes have been read.

**Figure 25: Sequential READ, Start from Random Location**